

Lucrarea nr. 4: **Implementarea funcțiilor binare folosind circuite logice de complexitate medie**

1. Scopul lucrării

În această lucrare se prezintă o serie de metode de implementare a funcțiilor binare ce necesită eforturi minime de simplificare. Aceste metode folosesc circuitele logice de complexitate medie precum decodificatoarele, demultiplexoarele, multiplexoarele sau memorii fixe de ROM sau EPROM. În partea aplicativă se urmărește fixarea deprinderilor acumulate în lucrările anterioare, privind dezvoltarea aplicațiilor cu structuri logice programabile de tip CPLD, fie pe baza schemelor logice, fie folosind limbajul VHDL.

2. Considerente teoretice

Realizarea funcțiilor binare numai cu ajutorul porților logice este, de cele mai multe ori, greoaie și implică un efort mare de calcul în vederea simplificării - mai ales când numărul variabilelor de intrare este mare. O soluție mai bună constă în utilizarea circuitelor de complexitate medie, în special a multiplexoarelor (MUX), a decodificatoarelor (DCD) sau a demultiplexoarelor DMUX.

2. 1. Utilizarea MUX în implementarea funcțiilor binare

Dacă implementăm o funcție logică direct după prima formă canonică (fără a face nici o simplificare), obținem o schemă logică structurată pe trei nivele:

- primul nivel este format din inversoare și este folosit pentru calculul complementelor variabilelor de intrare;
- nivelul doi este format din porți AND și este folosit pentru calculul mintermenilor funcției (reamintim că nu trebuie calculați toți mintermenii, este suficient să calculăm doar pe aceia pentru care funcția are valoarea "1");
- nivelul trei este format dintr-o singură poartă OR pentru a realiza sumarea mintermenilor.

Dacă analizăm structura internă a unui multiplexor, (vezi lucrarea anterioară), găsim o arhitectură similară cu cea rezultată în urma implementării după prima formă canonică. Diferențe apar pe nivelul doi: există câte o poartă AND pentru fiecare mintermen, în plus, fiecare poartă de pe acest nivel are o intrare suplimentară ce se constituie în intrarea de date a MUX.

Datorită acestor asemănări structurale a apărut destul de repede ideea că multiplexorul se poate folosi în implementarea de funcții binare. Intuitiv ne dăm seama că pe intrările de selecție trebuie conectate variabilele funcției ce trebuie implementate iar pe intrările de date valorile funcției. În funcție de raportul dintre numărul de variabile ale funcției binare și numărul de intrări de selecție ale MUX pot exista următoarele situații:

- A. Numărul intrărilor de selecție este egal cu numărul variabilelor funcției.** În acest caz, implementare funcției binare se face fără efort de minimizare și nu mai necesită nici un alt circuit suplimentar. Dacă conectarea variabilelor funcției la intrările de selecție se face cu respectarea ponderilor, ($x_0 \rightarrow A$, $x_1 \rightarrow B$, ...), nu trebuie să facem altceva decât să copiem valorile funcției din tabelul de adevăr pe intrările de date ale MUX.
- B. Numărul intrărilor de selecție este mai mic decât numărul variabilelor funcției.** În acest caz, o parte din variabilele funcției vor fi folosite pentru comanda intrărilor de selecție iar restul vor intra în calculul unor subfuncții binare ce se vor aplica la intrările de date ale MUX. Dacă numărul variabilelor funcției, x , este mai mare cu p față de numărul n al intrărilor de selecție ale MUX, atunci trebuie calculate 2^p funcții logice de $p = x - n$ variabile. Calculul celor n subfuncții binare necesită circuite suplimentare pe lângă MUX și necesită și un oarecare efort de simplificare. Oricum, cele n subfuncții sunt mult mai ușor de implementat decât funcția inițială deoarece au un număr mult mai mic de variabile de intrare.
- C. Numărul intrărilor de selecție este mai mare decât numărul variabilelor funcției.** Această situație corespunde unei utilizări neeficiente a MUX și nu prezintă un interes prea mare din punct de vedere practic. Totuși, dacă într-un anumit context suntem forțați să folosim așa ceva, se procedează astfel: variabilele funcției se conectează la intrările de selecție cu respectarea ponderilor; intrările de selecție nefolosite se conectează la masă; pe intrările de date se copiază valorile funcției; intrările de date nefolosite se conectează la masă.

Pentru o înțelegere mai bună a modului de utilizare a MUX-ului în implementarea funcțiilor binare sunt prezentate două exemple considerând că avem de implementat o funcție binară de trei variabile ce este dată prin tabelul de adevăr alăturat.

Exemplul 1: Se cere implementarea funcției F folosind un circuit MUX 8:1

Deoarece un MUX8:1 are trei intrări de selecție se constată că suntem într-o situație fericită în care numărul intrărilor de selecție este egal cu cel al variabilelor funcției.

Conectarea variabilelor funcției la intrările de selecție cu respectare ponderilor presupune următoarele conexiuni: $x_0 = A$, $x_1 = B$, $x_2 = C$.

x_2	x_1	x_0	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Din prelucrarea primei forme canonice (forma disjunctivă) a funcției F , se obține relația (1). Relația (2) nu este altceva decât funcția de transfer a multiplexorului 8:1.

$$F = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ \bar{x}_2 \bar{x}_1 x_0 \\ x_2 \bar{x}_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \\ x_2 x_1 \bar{x}_0 \\ x_2 x_1 x_0 \end{vmatrix} = \begin{vmatrix} 1 \bar{x}_2 \bar{x}_1 x_0 \\ 1 \bar{x}_2 x_1 \bar{x}_0 \\ 1 x_2 \bar{x}_1 \bar{x}_0 \\ 1 x_2 \bar{x}_1 x_0 \\ 1 x_2 x_1 \bar{x}_0 \\ 1 x_2 x_1 x_0 \end{vmatrix} = \begin{vmatrix} 0 \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ 1 \bar{x}_2 \bar{x}_1 x_0 \\ 1 \bar{x}_2 x_1 \bar{x}_0 \\ 0 \bar{x}_2 x_1 x_0 \\ 1 x_2 \bar{x}_1 \bar{x}_0 \\ 1 x_2 \bar{x}_1 x_0 \\ 0 x_2 x_1 \bar{x}_0 \\ 1 x_2 x_1 x_0 \end{vmatrix} = \begin{vmatrix} 0 S_0 \\ 1 S_1 \\ 1 S_2 \\ 0 S_3 \\ 1 S_4 \\ 1 S_5 \\ 0 S_6 \\ 1 S_7 \end{vmatrix} \quad (1)$$

$$Y = \begin{vmatrix} I_0 \bar{C} \bar{B} \bar{A} \\ I_1 \bar{C} \bar{B} A \\ I_2 \bar{C} B \bar{A} \\ I_3 \bar{C} B A \\ I_4 C \bar{B} \bar{A} \\ I_5 C \bar{B} A \\ I_6 C B \bar{A} \\ I_7 C B A \end{vmatrix} = \begin{vmatrix} I_0 S_0 \\ I_1 S_1 \\ I_2 S_2 \\ I_3 S_3 \\ I_4 S_4 \\ I_5 S_5 \\ I_6 S_6 \\ I_7 S_7 \end{vmatrix} \quad (2)$$

Din compararea relațiilor (1) și (2), prin identificare, rezultă: $I_0=0$, $I_1=1$, $I_2=1$, $I_3=0$, $I_4=1$, $I_5=1$, $I_6=0$, $I_7=1$. Schema de conectare a MUX 8:1 pentru a realizarea funcției cerute se prezintă în figura 1.a.

Conectarea variabilelor funcției la intrările de selecție în ordinea ponderilor ușurează modul de amplasare a valorilor funcției la intrările de date: valoare funcției pentru combinația 0 se conectează la I_0 , valoare funcției pentru combinația 1 se conectează la I_1 , etc.

Atenție: Conectarea fără respectarea ponderilor nu este interzisă însă, în astfel de situații, modul de conectare al valorilor funcției la intrările de date este cu totul altul.

Exemplul 2: Se cere implementarea funcției F folosind un circuit MUX 4:1

Deoarece un MUX 4:1 are două intrări de selecție se constată că numărul intrărilor de selecție este mai mic cu o unitate decât numărul de variabile ale funcției F.

Pentru astfel de cazuri, în mod arbitrar se alege două variabile ale funcției pentru comanda intrărilor de selecție ale multiplexorului. **În cazul de față alege următoarea variantă: $x_2 = A$, $x_0 = B$.**

Prelucrarea primei forme canonice (forma disjunctivă) a funcției F, se face în concordanță cu alegerea deja făcută și necesită o succesiune de câteva etape:

- scrierea primei forme canonice după tipicul deja cunoscut;
- rearanjarea variabilelor funcției în produse după ponderea intrărilor de selecție pe care le comandă (în cazul nostru, x_0 trebuie să apară pe prima poziție, x_2 pe poziția a doua după care vin restul de variabile);
- se dau factori comuni toate combinațiile posibile ale variabilelor ce comandă intrările de selecție (în cazul nostru, toate combinațiile posibile ale variabilelor $x_0 x_2$);
- rezultatul acestor prelucrări efectuate asupra formei canonice se compară cu funcția de transfer a MUX-ului folosit și de deduc expresiile logice ale subfuncțiilor ce trebuie conectate la intrările de date ale MUX (în cazul de față se compară relațiile 3 și 4).

$$F = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ \bar{x}_2 \bar{x}_1 x_0 \\ x_2 \bar{x}_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \\ x_2 x_1 \bar{x}_0 \\ x_2 x_1 x_0 \end{vmatrix} = \begin{vmatrix} x_0 \bar{x}_2 \bar{x}_1 \\ x_0 \bar{x}_2 x_1 \\ \bar{x}_0 x_2 \bar{x}_1 \\ x_0 x_2 \bar{x}_1 \\ x_0 \bar{x}_2 \bar{x}_1 \\ x_0 \bar{x}_2 x_1 \end{vmatrix} = \begin{vmatrix} \bar{x}_0 \bar{x}_2 x_1 \\ \bar{x}_0 x_2 \bar{x}_1 \\ x_0 \bar{x}_2 \bar{x}_1 \\ x_0 x_2 \bar{x}_1 \\ x_0 \bar{x}_2 x_1 \\ x_0 x_2 x_1 \end{vmatrix} = \begin{vmatrix} \bar{x}_0 \bar{x}_2 x_1 \\ \bar{x}_0 x_2 \bar{x}_1 \\ x_0 \bar{x}_2 \bar{x}_1 \\ x_0 x_2 1 \end{vmatrix} = \begin{vmatrix} S_0 x_1 \\ S_1 \bar{x}_1 \\ S_2 \bar{x}_1 \\ S_3 1 \end{vmatrix} \quad (3)$$

$$Y = \begin{vmatrix} I_0 \bar{B} \bar{A} \\ I_1 \bar{B} A \\ I_2 B \bar{A} \\ I_3 B A \end{vmatrix} = \begin{vmatrix} I_0 S_0 \\ I_1 S_1 \\ I_2 S_2 \\ I_3 S_3 \end{vmatrix} \quad (4)$$

Comparând relațiile (3) și (4) se constată că intrările multiplexorului trebuie conectate astfel: $I_0 = x_1$, $I_1 = \bar{x}_1$, $I_2 = \bar{x}_1$, $I_3 = 1$. Schema de conectare a MUX pentru realizarea funcției cerute se prezintă în figura 1. c).

O altă variantă de realizare a funcției binare F cu ajutorul unui MUX4:1, este prezentată în figura 1.b.

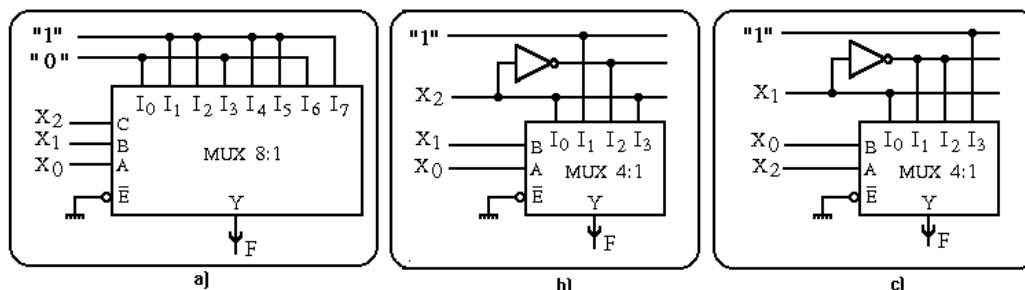


Fig. 1. Variante de implementare ale funcției F folosind multiplexoare de tip MUX8:1 respectiv MUX4:1.

• Concluzie

Multiplexorul poate fi privit ca un circuit universal pentru implementare funcțiilor logice dar prezintă inconvenientul că mintermenii (calculați la nivelul porților AND) nu pot fi utilizați decât o singură dată. În consecință, MUX-ul nu se pretează

pentru implementarea mai multor funcții binare de aceleași variabile deoarece nu există posibilitatea ca mintermenii calculați pentru o funcție să fie utilizați și de celelalte funcții.

2. 2. Utilizarea DCD în implementarea funcțiilor binare

Analizând schema logică a unui decodificator se observă că aceasta este organizată pe două nivele:

- un nivel de inversoare pentru calculul complementelor variabilelor de intrare;
- un nivel de NAND-uri (dacă ieșirile decodificatorului sunt active pe zero logic) sau un nivel de AND-uri (dacă ieșirile decodificatorului sunt active pe unu logic).

În ipoteza că pe intrările de selecție ale unui decodificator binar se aplică variabilele unei funcții binare, la ieșirile acestuia se vor regăsi mintermenii respectivei funcții (dacă DCD are ieșiri active pe unu logic) fie complementul mintermenilor (dacă DCD are ieșiri active pe zero logic).

Dacă variabilele funcției sunt conectate la intrările de selecție cu respectarea ponderilor, atunci numerotarea ieșirilor DCD-ului va fi identică cu numerotarea mintermenilor (m_0 este disponibil la ieșirea 0, m_1 este disponibil la ieșirea 1, ...).

Așadar, utilizarea DCD-ului este avantajoasă acolo unde este nevoie de implementarea mai multor funcții binare ce depind de aceleași variabile de intrare deoarece, mintermenii se calculează o singură dată și pot fi utilizați pentru fiecare funcție.

Regulile de conectare sunt următoarele:

- dacă în tabelul de adevăr funcția este definită cu mai puține stări de "1" decât stări de "0", atunci vom utiliza un NAND conectând la intrările lui mintermenii marcați cu "1";
- dacă în tabelul de adevăr funcția este definită cu mai puține stări de "0" decât stări de "1", atunci vom utiliza un AND conectând la intrările lui mintermenii marcați cu "0".

Pentru o înțelegere mai bună a modului de utilizare a unui DCD în implementarea funcțiilor binare, în cele ce urmează se exemplifică modul de conectare a ieșirilor DCD în funcție de tabelul de adevăr al funcției ce trebuie implementate. Ca exemplu se consideră funcția F dată prin tabelul de adevăr alăturat.

Exemplu 3: Se cere implementarea funcției folosind un DCD cu ieșiri active pe unu logic;

Deoarece DCD-ul are ieșirile active pe unu logic înseamnă că la ieșirile sale vom obține mintermenii funcției. Pentru rezolvarea acestei probleme vom scrie funcția F folosind formele canonice, după care, acestea vor fi prelucrate până la punerea în evidență a mintermenilor.

Prima formă canonică a funcției F are expresia:

$$F = \left| \begin{array}{c} \bar{x}_2 x_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \end{array} \right| = \left| \begin{array}{c} m_2 \\ m_5 \end{array} \right| \quad (5)$$

x_2	x_1	x_0	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

Relația (5) conduce la o schemă de implementare a funcției F ca cea din figura 2.a)

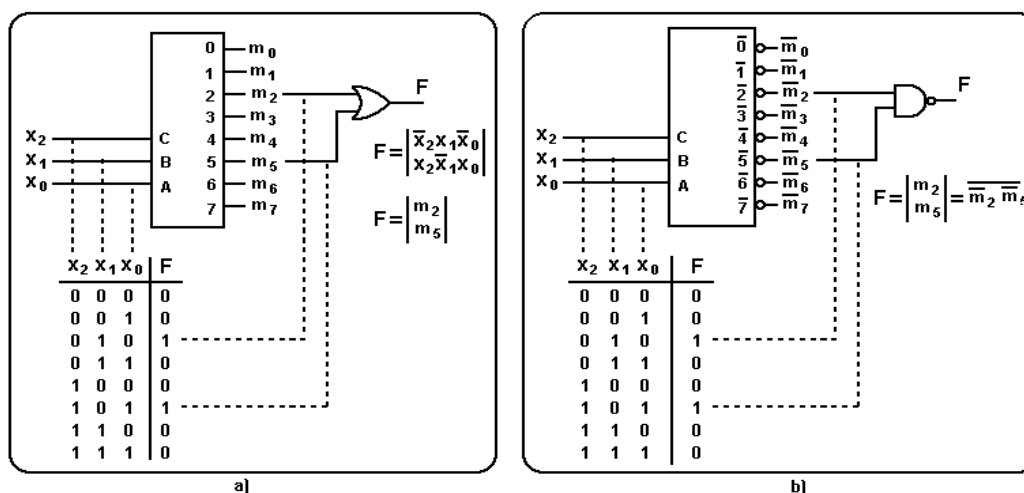


Fig. 2. Implementarea funcției F folosind circuite DCD

Exemplu 4: Se cere implementarea funcției folosind un DCD cu ieșiri active pe zero logic;

Deoarece DCD-ul are ieșirile active pe zero logic înseamnă că la ieșirile sale vom obține complementul mintermenii funcției. Pentru rezolvarea acestei probleme vom scrie funcția F folosind formele canonice, după care, acestea vor fi prelucrate până la punerea în evidență a complementelor mintermenilor.

Aplicând teoremele lui DeMorgan relația (5) se obține:

$$F = \left| \begin{array}{c} \bar{x}_2 x_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \end{array} \right| = \left| \begin{array}{c} m_2 \\ m_5 \end{array} \right| = \overline{\overline{\left| \begin{array}{c} m_2 \\ m_5 \end{array} \right|}} = \overline{\overline{m_2} \overline{m_5}} \quad (6)$$

Implementarea funcției după relația (6) este prezentată în figura 2.b).

2. 3. Utilizarea DMUX în implementarea funcțiilor binare

Este cunoscut faptul că un DMUX poate fi transformat ușor în DCD prin menținerea permanentă a intrării de validare în starea sa activă. Toate precizările făcute la utilizarea DCD în implementarea funcțiilor binare rămân valabile.

2. 4. Utilizarea memoriilor ROM în implementarea funcțiilor binare

Cea mai rapidă metodă de implementare a circuitelor combinaționale cu un număr mare de intrări și de ieșiri se bazează pe utilizarea memoriilor fixe de tip ROM (Read Only Memory), indiferent de variantele tehnologice de realizare ale acestora (ROM, PROM, EPROM sau EEPROM).

Această metodă nu necesită nici un efort de simplificare deoarece întreg tabelul de adevăr al circuitului combinațional este înscris în memoria nevolatilă. Modul de lucru este foarte simplu: liniile de adresă ale memoriei sunt comandate de către variabilele funcției/funcțiilor binare iar la ieșirile de date ale memoriei vor fi disponibile ieșirile funcției/funcțiilor binare ce au fost implementate.

Organizarea memoriei ROM este dependentă de circuitul combinațional ce trebuie implementat. Astfel: numărul liniilor de adresă trebuie să fie $\geq$ cu numărul variabilelor de intrare, iar numărul liniilor de date trebuie să fie $\geq$ numărul ieșirilor circuitului combinațional. Spre exemplu, cu ajutorul unei memorii EPROM cu organizarea $2K \times 8$, memorie caracterizată de 11 linii de adresă și 8 linii de date, se pot implementa simultan 8 funcții binare ce depind de cel mult 11 variabile de intrare.



3. Desfășurarea lucrării

3.1. Implementarea funcțiilor binare cu MUX.

A. Referitor la circuitele logice din fig. 1, răspundeți la următoarele întrebări:

- Cum trebuie conectate intrările de date ale MUX8:1, pentru a realiza aceeași funcție logică, în situația în care comanda intrărilor de selecție se face astfel: $x_0 \rightarrow C, x_1 \rightarrow B, x_2 \rightarrow A$;
- Cum trebuie conectate intrările de date ale MUX4:1, pentru a realiza aceeași funcție logică, în situația în care comanda intrărilor de selecție se face astfel: $x_1 \rightarrow B, x_2 \rightarrow A$;

B. Folosind modul de lucru prezentat în exemplul 2, se cere implementarea unei funcții logice cu tabelul de adevăr alăturat, folosind un MUX 4:1.

c	b	a	yout
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

3.2. Implementarea funcțiilor binare cu DCD.

A. Am arătat anterior că DCD este util în implementarea circuitelor logice, mai ales acolo unde avem mai multe funcții binare ce depind de aceleași variabile de intrare. Un astfel de exemplu este și decodificatorul BCD-7segmente. Pentru un astfel de circuit se cer:

- tabelul de adevăr astfel încât să afișeze caracterele din figura alăturată (unu logic = segment aprins);
- implementarea funcțiilor $a \div g$ folosind un DCD cu ieșiri active pe zero logic și un număr cât mai redus de porți logice auxiliare;
- implementarea funcțiilor $a \div g$ folosind un DCD cu ieșiri active pe zero logic și un număr cât mai redus de porți logice auxiliare;



B. Folosind modul de lucru prezentat în exemplul 3, se cere implementarea unei funcții logice cu tabelul de adevăr de la punctul 3.1.B., folosind un DCD cu ieșiri active pe zero logic;

3.3. Utilizarea ISE WebPack pentru descrierea aplicațiilor sub formă de scheme logice

- A. Implementarea și verificarea pe macheta de laborator cu CPLD a schemelor logice din figura 1.
- B. Implementarea și verificarea pe macheta de laborator cu CPLD a schemelor logice din figura 2.
- C. Implementarea și verificarea pe macheta de laborator cu CPLD a schemei logice rezultate la punctul 3.1.B.
- D. Implementarea și verificarea pe macheta de laborator cu CPLD a schemei logice rezultate la punctul 3.2.A.

Mod de lucru: vezi indicațiile de la "Implementarea funcțiilor binare cu rețele de porți logice"

3.4. Aplicații cu VHDL.

- A. Realizați o descriere comportamentală, în limbaj VHDL, pentru circuitele logice din figura 2, după care verificați funcționarea acestora pe macheta de laborator cu CPLD.
- B. Realizați o descriere în limbaj VHDL și implementați pe macheta cu CPLD, un circuit de transcodare pe 4 biți care să facă trecerea de la coul binar natural la codul Gray.

